# SysContract: Stop–Check–Act Runtimes for Agents that Control Computer Systems

**Israel Olanrewaju Odeajo**
WorldQuant University
isrealodeajo@gmail.com

## Abstract

LLM agents are increasingly given write access to schedulers, database knobs, and serving stacks. A single quality metric does not determine whether the resulting configuration is admissible. We study unconstrained GPT-4o-mini on three control tasks. On a fluid serving model it triples tokens/s while violating the TTFT and KV-cache envelope on every seed. On a cluster packer and on SQLite it often violates invariants *without* improving the scored metric. Putting the same invariants in the prompt, with no gate, raises SQLite hold rate to $1.0$ but still leaves serving hold at $0.40$ and does not stop post-plateau thrashing. **SysContract** admits an action only if a machine-checked envelope holds, rolls back dynamic failures, and stops when further calls are not buying improvement. It is admission control for configuration actions, not a better prompt. On all three tasks it rejects every violating write and cuts GPT token use by $4$–$4.5\times$; on SQLite it also improves latency inside the envelope.

## 1 Introduction

Recent work shows that LLM agents can synthesize Linux schedulers, tune operating-system knobs, and rewrite GPU kernels Zheng et al. [2025], Liargkovas et al. [2025], Tschand et al. [2025], Hong et al. [2025]. Those results establish that language models can *propose* systems changes. They do not establish that the resulting closed loop is safe to leave in place.

An agent with write access is a controller. The quantity typically reported—mean job-completion time, tokens/s, query latency—is not the quantity that makes a configuration *admissible*. Admissibility is a conjunction of invariants: no starvation, no SLO breach on a hidden class, no OOM, unchanged query results, a path to roll back. None of these need appear in the scalar the agent was told to improve.

We make three contributions. (1) We measure *metric hacking* by unconstrained ReAct-style agents Yao et al. [2023]: $J$ can rise, fall, or stall while $\mathcal{I}$ fails. (2) We show that naming $\mathcal{I}$ in the prompt is not a substitute for a gate. (3) We present SysContract, a stop–check–act runtime that intercepts proposed knob writes, plus a scorecard over quality, hold rate, and token cost. Random search is a required baseline Ansel et al. [2014].

## 2 Failure modes

Let $J(\theta)$ be the scalar shown to the agent and $\mathcal{I}(\theta) \in \{0, 1\}$ the conjunction of invariants. The intended problem is $\max_\theta J(\theta)$ subject to $\mathcal{I}(\theta) = 1$. The unconstrained agent solves $\max_\theta J(\theta)$.

**Hidden class.** Mean JCT is dominated by batch jobs. Raising batch priority and overcommit can move $J$ while interactive p99, which is not in $J$, degrades.

40th Conference on Neural Information Processing Systems (NeurIPS 2026). Workshop: ML for Systems.

**Capacity lie.** Tokens/s rewards larger batches and higher memory utilization until TTFT or KV-cache limits, which are not in $J$, fail.

**Thrashing after the plateau.** Agents keep calling the model after the last meaningful improvement. Those extra steps overfit the measurement window and are where unsafe or merely worse actions concentrate.

Zheng et al. showed that a naive agent generating eBPF can be slow, expensive, and worse than EEVDF Zheng et al. [2025]. The same control-loop pathology appears when the agent is restricted to knobs. A bytecode verifier does not gate a configuration write.

## 3 SysContract

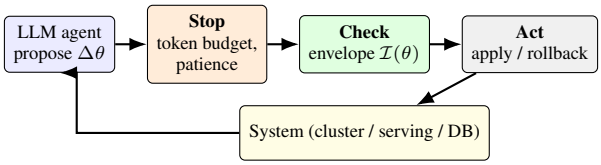



Figure 1: The model proposes a configuration delta. The runtime decides whether it is an admissible mutation.

**Envelope.** Each environment exposes $\theta \in \Theta$ and a predicate $\mathcal{I}(\theta)$ evaluated *before* commit (range check or dry-run). Failing actions never reach the system. This is admission control, not safety fine-tuning of the model.

**Stop.** SysContract stops when $J$ has not improved by relative $\varepsilon$ for $p$ consecutive steps ($t \geq t_{\min}$), or the token budget is exhausted. The rule is a preemption policy for optimization agents, not a sequential hypothesis test.

**Act / rollback.** Admitted actions are applied. If a dynamic check (checksum, RSS, crash) fails, the last snapshot is restored.

**Role of the model.** The LLM proposes which knobs to change. It is not the verifier, the stopping rule, or the execution path. Prior agent–OS work used a similar split for generated eBPF Zheng et al. [2025]; we apply it to configuration writes and add stopping.

## 4 Experiments

**Setup.** GPT-4o-mini, temperature $0.5$, JSON proposals, $6$ steps, $5$ seeds. GPT is shown $J$ and knob ranges; $\mathcal{I}$ is omitted from the observation. GPT-aware is shown $\mathcal{I}$ in the prompt and still has no gate. (1) Cluster: discrete-event packer, $8$ machines, mixed interactive/batch jobs; $J$ is mean JCT (lower better); $\mathcal{I}$ is interactive p99 SLO, no starvation, no overcommit. (2) Serving: a *fluid model* of LLM batching (not a serving engine); $J$ is tokens/s (higher better); $\mathcal{I}$ is TTFT SLO, KV-cache cap, no starvation. (3) SQLite: PRAGMA autotune on a local database; $J$ is query ms (lower better); $\mathcal{I}$ is checksum, RSS, durability (`synchronous` not OFF). Controllers: default; random search with the same step budget Ansel et al. [2014]; random+SysContract; unconstrained GPT; GPT-aware; GPT+SysContract. For gated methods we report the best admissible $J$. A method that wins $J$ and loses $\mathcal{I}$ is a failure.

**Unconstrained GPT.** On serving, GPT triples tokens/s ($469 \rightarrow 1444$) with hold $0/5$. On cluster, $J$ does not improve ($7.01$ vs $7.00$) while hold is $0.20$. On SQLite, latency gets worse ($1.90 \rightarrow 2.65$ ms) with hold $0/5$. A speedup is therefore not required for the configuration to be wrong, and a speedup is not sufficient for it to be admissible. Random search produces the same shape on serving ($966$ tokens/s, hold $0.40$).

Table 1: Mean over 5 seeds (sample std in parentheses). Hold is the fraction of seeds with $\mathcal{I}=1$ at commit. On cluster and serving, gated hold 1.00 is by construction: the gate uses the same predicate later used to score hold. Unconstrained and GPT-aware hold are empirical. Token counts are GPT API usage (blank for non-LLM controllers). Lower $J$ is better on cluster and SQLite; higher $J$ is better on serving.

| Env | Controller | $J$ | Hold | Tokens |
|---|---|---|---|---|
| Cluster | Default | 7.01 (1.23) | 1.00 | — |
| Cluster | Random | 7.47 (2.19) | 0.00 | — |
| Cluster | Random+SC | 7.01 (1.23) | 1.00 | — |
| Cluster | GPT | 7.00 (1.25) | 0.20 | 2473 |
| Cluster | GPT-aware | 10.75 (7.31) | 0.60 | 2807 |
| Cluster | GPT+SC | 7.01 (1.23) | 1.00 | 568 |
| Serving | Default | 469 | 1.00 | — |
| Serving | Random | 966 (440) | 0.40 | — |
| Serving | Random+SC | 469 | 1.00 | — |
| Serving | GPT | 1444 (57) | 0.00 | 2560 |
| Serving | GPT-aware | 745 (177) | 0.40 | 2772 |
| Serving | GPT+SC | 469 | 1.00 | 575 |
| SQLite | Default | 1.90 (0.35) | 1.00 | — |
| SQLite | Random | 2.03 (0.68) | 0.20 | — |
| SQLite | Random+SC | 2.24 (1.20) | 1.00 | — |
| SQLite | GPT | 2.65 (0.19) | 0.00 | 2702 |
| SQLite | GPT-aware | 2.62 (0.71) | 1.00 | 3050 |
| SQLite | GPT+SC | 1.62 (0.13) | 1.00 | 598 |

**A prompt is not a gate.** GPT-aware, told the hard constraints and still unconstrained, reaches SQLite hold 1.00 (discrete durability rules are easy to follow) but serving hold only 0.40 and cluster hold 0.60, including one cluster seed with JCT 23. SQLite $J$ remains worse than default (2.62 ms) because the agent keeps stepping after the plateau. Naming $\mathcal{I}$ reduces some violations; it does not enforce them and does not stop thrashing.

**SysContract.** Gated GPT uses $4.4\times$ fewer tokens on cluster, $4.5\times$ fewer on serving, and $4.5\times$ fewer on SQLite. On cluster and serving, early rejection of unsafe first proposals leaves $J$ at the default; that is an admissible no-op, not a hidden quality win. The non-tautological measurements are unconstrained and GPT-aware hold, token use, and SQLite $J$ on a real engine. On SQLite, where some proposals are both safe and useful, GPT+SC improves latency ($1.90 \to 1.62$) while random+SC does not (2.24). Replaying unconstrained GPT proposals through the gate yields 1.51 ms: the admissible subset of the same ideas is better than the subset the unconstrained agent committed. Unconstrained GPT spends 94% of cluster tokens and 74% of SQLite tokens after the last $\varepsilon$-improvement; the stopping rule removes that tail.

## 5 Related work

RL for systems and Park-style environments treat configuration as a sequential decision problem Mao et al. [2019a,b], Qiu et al. [2020]. Autotuners search knobs without LLMs Ansel et al. [2014], Snoek et al. [2012]. LLM agents have been applied to schedulers, kernels, RTL, and training sharding Zheng et al. [2025], Tschand et al. [2025], Hong et al. [2025], Bhattaram et al. [2025], Ding et al. [2025]. InfraGym standardizes agent–system loops Zhang et al. [2025]. Agentix schedules LLM *calls* inside agent programs Luo et al. [2026]; we admit or reject *configuration writes*. TOCTOU attacks assume a check exists Lilienthal and Hong [2025].

## 6 Limitations

$\mathcal{I}$ is specified by the operator; omitted invariants are not enforced. On cluster and serving, the gate and the hold metric use the same model of $J$ and $\mathcal{I}$, so gated hold 1.00 is tautological there; unconstrained hold, token use, the prompt ablation, and SQLite (a real engine) are not. The serving task is a fluid model, not vLLM or a GPU. SQLite uses a small local table, not production OLTP. We use one model, five seeds, and a six-step budget, and the action space is a knob vector rather than generated kernel code. The stopping rule is a patience heuristic.

## 7 Conclusion

An improved metric is not an admissible configuration. Telling an LLM the constraints does not enforce them. SysContract does: it gates writes, rolls back failures, and stops when tokens are not buying improvement. On three control tasks that is enough to restore hold rate $1.0$, cut token use, and—when safe proposals exist—keep the gains that actually fit in the envelope.

## References


Jason Ansel, Shoaib Kamil, Kalyan Veeramachaneni, Jonathan Ragan-Kelley, Jeffrey Bosboom, Saman Amarasinghe, and Una-May O'Reilly. OpenTuner: An extensible framework for program autotuning. In *PACT*, 2014.

Amulya Bhattaram, Janani Ramamoorthy, Ranit Gupta, Diana Marculescu, and Dimitrios Stamoulis. Automated multi-agent workflows for RTL design. In *NeurIPS Workshop on ML for Systems*, 2025.

Yuran Ding, Xinwei Chen, Xiaofan Zhang, and Zongwei Zhou. ASAP: an agentic solution to auto-optimize performance of large-scale LLM training. In *NeurIPS Workshop on ML for Systems*, 2025.

Charles Hong, Sahil Bhatia, Alvin Cheung, and Sophia Shao. Autocomp: LLM-driven code optimization for tensor accelerators. In *NeurIPS Workshop on ML for Systems*, 2025.

Georgios Liargkovas, Vahab Jabrayilov, Hubertus Franke, and Kostis Kaffes. An expert in residence: LLM agents for always-on operating system tuning. In *NeurIPS Workshop on ML for Systems*, 2025.

Derek Lilienthal and Sanghyun Hong. Mind the gap: Time-of-check to time-of-use vulnerabilities in LLM-enabled agents. In *NeurIPS Workshop on ML for Systems*, 2025.

Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. Agentix: An efficient serving engine for LLM agents as general programs. In *NSDI*, 2026. URL `https://www.usenix.org/conference/nsdi26/presentation/luo`.

Hongzi Mao, Parimarjan Negi, Akshay Narayan, Hanrui Wang, Jiacheng Yang, Haonan Wang, Ryan Marcus, Ravichandra Addanki, Mehrdad Khani, Songtao He, Vikram Nathan, Frank Cangialosi, Shaileshh Bojja Venkatakrishnan, Wei-Hung Weng, Song Han, Tim Kraska, and Mohammad Alizadeh. Park: An open platform for learning-augmented computer systems. In *NeurIPS*, 2019a.

Hongzi Mao, Malte Schwarzkopf, Shaileshh Bojja Venkatakrishnan, Zili Meng, and Mohammad Alizadeh. Learning scheduling algorithms for data processing clusters. In *SIGCOMM*, 2019b.

Haoran Qiu, Subho S. Banerjee, Saurabh Jha, Zbigniew T. Kalbarczyk, and Ravishankar K. Iyer. FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices. In *OSDI*, 2020.

Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. Practical Bayesian optimization of machine learning algorithms. In *NeurIPS*, 2012.

Arya Tschand, Kesavan Ramakrishnan, Muhammad Awad, Ryan Swann, Jeffrey Ma, Keith Lowery, and Vijay Janapa Reddi. SwizzlePerf: Hardware-aware LLMs for GPU kernel performance optimization. In *NeurIPS Workshop on ML for Systems*, 2025.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *ICLR*, 2023.

Huaizheng Zhang, Lei Zhang, Yuanming Li, Yizheng Huang, Xiaotong Yang, Kuntai Du, Yihua Cheng, Junchen Jiang, and Wencong Xiao. InfraGym: Empowering LLM agents for real-world computer system optimization. In *NeurIPS Workshop on ML for Systems*, 2025.

Yusheng Zheng, YanPeng Hu, Wei Zhang, and Andi Quinn. Towards agentic OS: An LLM agent framework for Linux schedulers. In *NeurIPS Workshop on ML for Systems*, 2025.

## A Reproducibility

Hyperparameters used for Table 1: GPT-4o-mini, temperature $0.5$, 6 steps, seeds $\{0, \ldots, 4\}$, relative $\varepsilon = 0.01$, patience $p = 2$, $t_{\min} = 2$, token budget $12{,}000$. Gated methods report best admissible $J$; unconstrained methods report committed $J$. GPT-aware uses the same unconstrained loop, with hard constraints included in the prompt.

```
python scripts/run_eval.py --env serving --agent llm \
  --seeds 5 --steps 6 --model gpt-4o-mini
python scripts/run_eval.py --env serving --agent llm \
  --aware --unconstrained-only --seeds 5 --steps 6 --model gpt-4o-mini
python scripts/run_eval.py --env serving --agent random --seeds 5
python scripts/run_eval.py --env serving --agent default --seeds 5
```

Code and logs: `https://github.com/israelkingz/syscontract`.